

Компьютерная графика

Лекция 5: Bind groups, uniform буферы, текстуры, сэмплеры

2025

Как мы умеем передавать данные на GPU?

- Immediates – ограничен размер

Как мы умеем передавать данные на GPU?

- Immediates – ограничен размер
- Атрибуты вершин, индексы – ограничены способы использования (только через растеризацию, строгий формат, нет доступа к соседним вершинам)

Как мы умеем передавать данные на GPU?

- Immediates – ограничен размер
- Атрибуты вершин, индексы – ограничены способы использования (только через растеризацию, строгий формат, нет доступа к соседним вершинам)
- Хочется уметь передавать больше произвольных данных

Как мы умеем передавать данные на GPU?

- Immediates – ограничен размер
- Атрибуты вершин, индексы – ограничены способы использования (только через растеризацию, строгий формат, нет доступа к соседним вершинам)
- Хочется уметь передавать больше произвольных данных
- $\implies$ *bind группы*

- Bind group – контейнер объектов (*bindings*), которые можно передать на GPU в шейдер

- Bind group – контейнер объектов (*bindings*), которые можно передать на GPU в шейдер
- Могут содержать:
 - Буферы: *uniform, storage, read-only storage*

- Bind group – контейнер объектов (*bindings*), которые можно передать на GPU в шейдер
- Могут содержать:
 - Буферы: *uniform*, *storage*, *read-only storage*
 - Текстуры (read-only изображения в памяти GPU)

- Bind group – контейнер объектов (*bindings*), которые можно передать на GPU в шейдер
- Могут содержать:
 - Буферы: *uniform, storage, read-only storage*
 - Текстуры (read-only изображения в памяти GPU)
 - Сэмплеры (настройки чтения изображений)

- Bind group – контейнер объектов (*bindings*), которые можно передать на GPU в шейдер
- Могут содержать:
 - Буферы: *uniform, storage, read-only storage*
 - Текстуры (read-only изображения в памяти GPU)
 - Сэмплеры (настройки чтения изображений)
 - Storage текстуры (read-write изображения)

- Объекты в bind группе описываются по числовым индексам

Bind groups

- Объекты в bind группе описываются по числовым индексам
- Пример bind-группы:

```
binding 0: uniform-буфер с параметрами камеры  
binding 1: environment-текстура для отражений  
binding 2: sampler для чтения текстуры отражений  
binding 3: буфер с описанием источников света
```

Bind group layout

- Для создания bind group нужно сначала описать *bind group layout*

Bind group layout

- Для создания bind group нужно сначала описать *bind group layout*
- Layout описывает, объекты какого типа (буфер/текстура/etc) будут храниться в bind группе (без ссылки на конкретные объекты), и в каких шейдерах они будут использоваться (вершинный/фрагментный)

Bind group layout

- Для создания bind group нужно сначала описать *bind group layout*
- Layout описывает, объекты какого типа (буфер/текстура/etc) будут храниться в bind группе (без ссылки на конкретные объекты), и в каких шейдерах они будут использоваться (вершинный/фрагментный)
- Пример bind group layout'a:

```
binding 0: uniform-буфер  
binding 1: 2D текстура  
binding 2: sampler  
binding 3: storage буфер
```

- При создании render pipeline нужно указать, какие ресурсы нужны этому pipeline для рисования

Pipeline layout

- При создании render pipeline нужно указать, какие ресурсы нужны этому pipeline для рисования
- Это делается через создание *pipeline layout*: он описывает набор bind group layout'ов, которые ему нужны

Pipeline layout

- При создании render pipeline нужно указать, какие ресурсы нужны этому pipeline для рисования
- Это делается через создание *pipeline layout*: он описывает набор bind group layout'ов, которые ему нужны
- Сами bind группы внутри pipeline layout тоже описываются индексами

Pipeline layout

- При создании render pipeline нужно указать, какие ресурсы нужны этому pipeline для рисования
- Это делается через создание *pipeline layout*: он описывает набор bind group layout'ов, которые ему нужны
- Сами bind группы внутри pipeline layout тоже описываются индексами
- Пример pipeline layout'a:

```
group 0: настройки камеры и освещения  
group 1: параметры материала объекта  
group 2: вспомогательные текстуры (шум, дизеринг)
```

- *Bind group / pipeline layout* описывает формат

- *Bind group / pipeline layout* описывает формат
- *Bind group* ссылается на конкретные данные (буфер/текстура/etc)

Bind groups + pipelines вместе

- *Bind group / pipeline layout* описывает формат
- *Bind group* ссылается на конкретные данные (буфер/текстура/etc)
- *Render pipeline* описывает конкретный конвейер рендеринга (шейдеры + настройки примитивов + etc)

- Разные bind группы могут использовать один и тот же bind group layout: например, layout материала объекта (набор текстур), у каждого объекта своя bind группа с конкретными текстурами

Bind groups + pipelines вместе

- Разные bind группы могут использовать один и тот же bind group layout: например, layout материала объекта (набор текстур), у каждого объекта своя bind группа с конкретными текстурами
- Разные render pipelines могут использовать один и тот же pipeline layout: например, ландшафт и 3D модели используют разные шейдеры, но один и тот же набор типов входных текстур и буферов (один и тот же *layout*)

Bind groups + pipelines: пример

Bind group layout настроек камеры и освещения

binding 0: uniform-буфер

binding 1: storage-буфер

Bind group с настройками камеры и освещения

binding 0: uniform-буфер с параметрами камеры

binding 1: storage-буфер с источниками света

Bind group layout описания материала объекта

binding 0: 2D текстура

binding 1: 2D текстура

binding 2: 2D текстура

Bind group с описанием материала объекта

binding 0: текстура альbedo (цвет)

binding 1: текстура нормалей

binding 2: текстура материала (metallic-roughness)

Pipeline layout:

group 0: настройки камеры и освещения

group 1: описание материала объекта

Bind groups в шейдере

```
struct Camera {  
    viewProjection: mat4x4f,  
    position: vec3f,  
    zRange: vec2f,  
}  
  
struct PointLight {  
    position: vec3f,  
    intensity: vec3f,  
    falloff: vec3f,  
}  
  
@group(0) @binding(0) var<uniform> camera : Camera;  
@group(0) @binding(1) var<storage, read> lights : array<PointLight>;  
  
@group(1) @binding(0) var albedo_texture : texture_2d<f32>;  
@group(1) @binding(1) var normal_texture : texture_2d<f32>;  
@group(1) @binding(2) var material_texture : texture_2d<f32>;  
@group(1) @binding(3) var linear_mipmap_sampler : sampler;
```

Bind groups + pipelines: рендеринг

```
render_pass.set_pipeline(my_pipeline)
render_pass.set_bind_group(0, camera_bind_group)

for object : objects {
    render_pass.set_bind_group(1, object.material_bind_group)
    render_pass.set_vertex_buffer(0, object.vertex_buffer)
    render_pass.set_index_buffer(object.index_buffer)
    render_pass.draw_indexed(object.index_count)
}
```

- Буферы, данные которых можно передать как read-only в шейдеры

- Буферы, данные которых можно передать как read-only в шейдеры
- Uniform-данные гарантированно не могут меняться в рамках одной команды рисования (отсюда название *uniform*)

- Буферы, данные которых можно передать как read-only в шейдеры
- Uniform-данные гарантированно не могут меняться в рамках одной команды рисования (отсюда название *uniform*)
- Для них у GPU есть специальные кеши памяти и отдельные (“скалярные”) инструкции для их чтения

- Буферы, данные которых можно передать как read-only в шейдеры
- Uniform-данные гарантированно не могут меняться в рамках одной команды рисования (отсюда название *uniform*)
- Для них у GPU есть специальные кеши памяти и отдельные (“скалярные”) инструкции для их чтения
- $\implies$ Работают быстро :)

- Такой буфер нужно создать с флагом `Uniform`

- Такой буфер нужно создать с флагом `Uniform`
- Тип в bind group layout: `buffer binding type = uniform`

Uniform буферы

- Такой буфер нужно создать с флагом `Uniform`
- Тип в `bind group layout`: `buffer binding type = uniform`
- Есть ограничения на размер буфера (точнее, его части), которую можно привязать к `bind` группе (обычно 64 Кб)

Uniform буферы vs immediates

- У uniform-буфера размер гораздо больше, чем у immediates (в 512 раз!), зачем использовать immediates?

Uniform буферы vs immediates

- У uniform-буфера размер гораздо больше, чем у immediates (в 512 раз!), зачем использовать immediates?
- Uniform-буфер – разновидность буфера, загрузить в него данные можно только через очередь `queue.write_buffer` (либо через асинхронный mapping, но там те же проблемы)

Uniform буферы vs immediates

- У uniform-буфера размер гораздо больше, чем у immediates (в 512 раз!), зачем использовать immediates?
- Uniform-буфер – разновидность буфера, загрузить в него данные можно только через очередь `queue.write_buffer` (либо через асинхронный mapping, но там те же проблемы)
- Последовательность команд на рисование отправляется в очередь целыми render pass'ами

Uniform буферы vs immediates

- У uniform-буфера размер гораздо больше, чем у immediates (в 512 раз!), зачем использовать immediates?
- Uniform-буфер – разновидность буфера, загрузить в него данные можно только через очередь `queue.write_buffer` (либо через асинхронный mapping, но там те же проблемы)
- Последовательность команд на рисование отправляется в очередь целыми render pass'ами
- $\Rightarrow$ Нельзя обновить данные uniform-буфера в середине render pass'a

Uniform буферы vs immediates

- У uniform-буфера размер гораздо больше, чем у immediates (в 512 раз!), зачем использовать immediates?
- Uniform-буфер – разновидность буфера, загрузить в него данные можно только через очередь `queue.write_buffer` (либо через асинхронный mapping, но там те же проблемы)
- Последовательность команд на рисование отправляется в очередь целыми render pass'ами
- $\implies$ Нельзя обновить данные uniform-буфера в середине render pass'a
- $\implies$ Обычно в uniform буферах хранят данные, постоянные в течение одного кадра: параметры камеры, настройки глобального освещения

Uniform буферы: dynamic offset

- Есть ограниченная поддержка динамического обновления uniform-буфера в виде *dynamic offsets*

Uniform буферы: dynamic offset

- Есть ограниченная поддержка динамического обновления uniform-буфера в виде *dynamic offsets*
- Можно включить dynamic offsets для конкретного буфера в bind group layout: `has_dynamic_offset = true`

Uniform буферы: dynamic offset

- Есть ограниченная поддержка динамического обновления uniform-буфера в виде *dynamic offsets*
- Можно включить dynamic offsets для конкретного буфера в bind group layout: `has_dynamic_offset = true`
- Функция `render_pass.set_bind_group()` принимает массив *dynamic offsets* – по одному на каждый uniform-буфер в этой группе, для которых включен dynamic offset

Uniform буферы: dynamic offset

- Есть ограниченная поддержка динамического обновления uniform-буфера в виде *dynamic offsets*
- Можно включить dynamic offsets для конкретного буфера в bind group layout: `has_dynamic_offset = true`
- Функция `render_pass.set_bind_group()` принимает массив *dynamic offsets* – по одному на каждый uniform-буфер в этой группе, для которых включен dynamic offset
- Тогда будет читаться кусок буфера, начинающийся с этого offset'a

Uniform буферы: dynamic offset

- Есть ограниченная поддержка динамического обновления uniform-буфера в виде *dynamic offsets*
- Можно включить dynamic offsets для конкретного буфера в bind group layout: `has_dynamic_offset = true`
- Функция `render_pass.set_bind_group()` принимает массив *dynamic offsets* – по одному на каждый uniform-буфер в этой группе, для которых включен dynamic offset
- Тогда будет читаться кусок буфера, начинающийся с этого offset'a
- Сами offset'ы должны быть кратны значению `minUniformBufferOffsetAlignment` (по умолчанию – 256)

Uniform буферы: dynamic offset

- Есть ограниченная поддержка динамического обновления uniform-буфера в виде *dynamic offsets*
- Можно включить dynamic offsets для конкретного буфера в bind group layout: `has_dynamic_offset = true`
- Функция `render_pass.set_bind_group()` принимает массив *dynamic offsets* – по одному на каждый uniform-буфер в этой группе, для которых включен dynamic offset
- Тогда будет читаться кусок буфера, начинающийся с этого offset'a
- Сами offset'ы должны быть кратны значению `minUniformBufferOffsetAlignment` (по умолчанию – 256)
- $\implies$ Можно загрузить несколько наборов данных в uniform буфер и динамически переключаться между ними

Uniform буферы: шейдер

```
struct Camera {  
    viewProjection: mat4x4f,  
}  
  
@group(0) @binding(0) var<uniform> camera : Camera;  
  
struct Immediates {  
    model: mat4x4f,  
}  
  
var<immediate> immediates;  
  
@vertex  
fn vertex_main(in: VertexIn) -> @builtin(position) vec4f {  
    return camera.viewProjection  
        * immediates.model  
        * vec4f(in.position, 1.0);  
}
```

Uniform буферы: инициализация

```
let bind_group_layout = device.create_bind_group_layout([
  BindGroupLayoutEntry {
    binding: 0,
    visibility: VERTEX,
    type: Buffer { type: Uniform },
  },
])

let uniform_buffer = device.create_buffer({
  size: 64,
  usage: CopyDst | Uniform,
})

let bind_group = device.create_bind_group(bind_group_layout, [
  BindGroupEntry {
    binding: 0,
    buffer: uniform_buffer,
  },
])
```

Uniform буферы: рендеринг

```
queue.write_buffer(uniform_buffer, &camera_matrix)

...

render_pass.set_bind_group(0, bind_group)
render_pass.set_pipeline(...)
render_pass.set_immediates(...)
render_pass.draw(...)

...

queue.submit([command_buffer.finish()])
```

- Текстуры – изображения в памяти GPU (чаще – несколько изображений)

- Текстуры – изображения в памяти GPU (чаще – несколько изображений)
- Их можно читать из шейдеров

- Текстуры – изображения в памяти GPU (чаще – несколько изображений)
- Их можно читать из шейдеров
- В них можно загружать пиксели из CPU

- Текстуры – изображения в памяти GPU (чаще – несколько изображений)
- Их можно читать из шейдеров
- В них можно загружать пиксели из CPU
- В них можно рисовать (вместо рисования в окно – про это в следующий раз)

- Текстуры – изображения в памяти GPU (чаще – несколько изображений)
- Их можно читать из шейдеров
- В них можно загружать пиксели из CPU
- В них можно рисовать (вместо рисования в окно – про это в следующий раз)
- Поддерживают много специфичных для изображений операций

- 1D: одномерная текстура (полоска пикселей)

Одномерная текстура

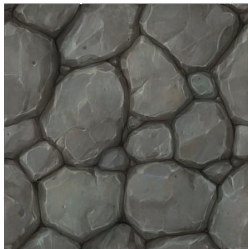
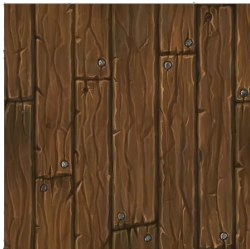
- Градиенты цветов
- Предподсчитанные значения функций (напр. для освещения)



- 1D: одномерная текстура (полоска пикселей)

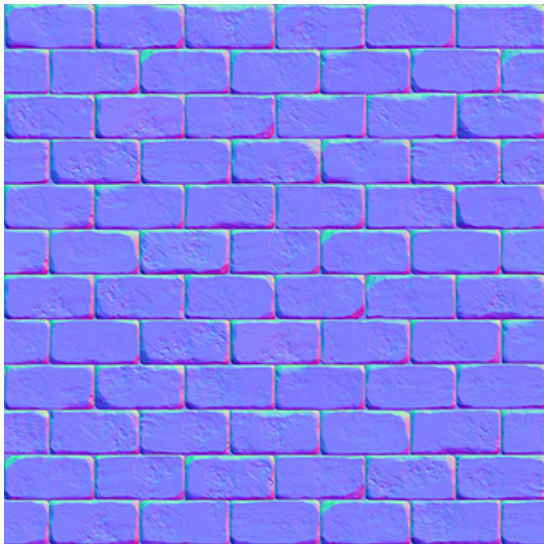
- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура

Двумерная текстура

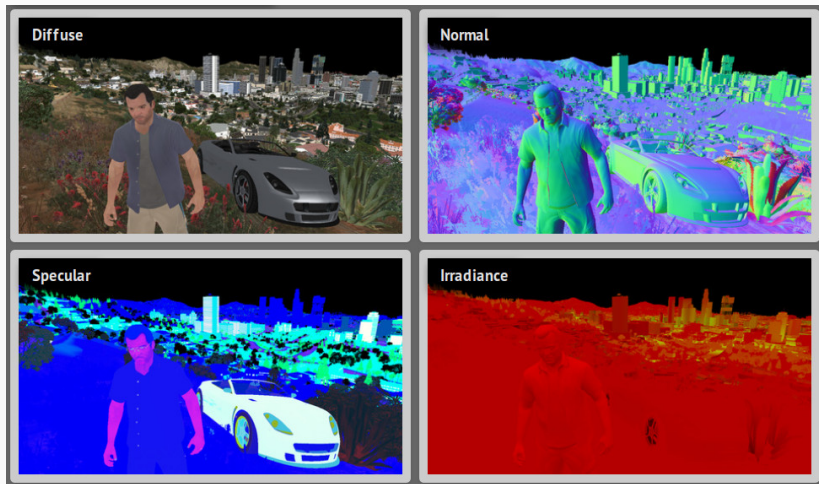


Двумерная текстура

- Самый часто используемый тип
- Любые числовые свойства поверхностей: цвет (альбедо), нормаль, тип материала
- Промежуточные изображения при рендеринге: G-buffer, ID (visibility) buffer, HDR buffer
- Карта высот ландшафта
- Элементы UI
- Предподсчитанные значения 2D функции
- etc.



G-buffer

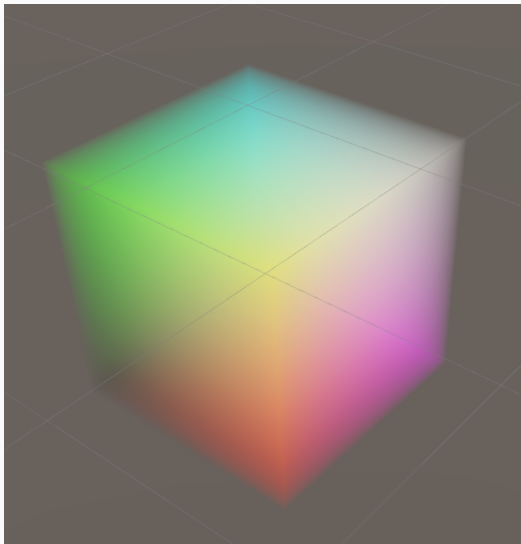




- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура

- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура
- 3D: трёхмерная текстура

Трёхмерная текстура



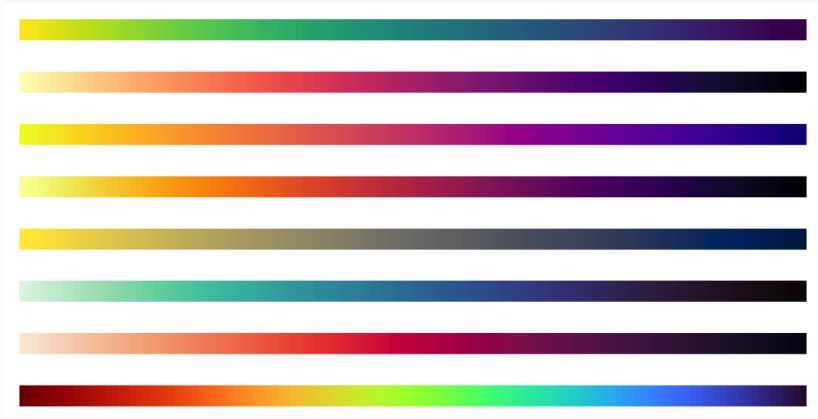
- Любые числовые свойства пространства: плотность, цвет, коэффициент рассеяния
- Объёмные (volumetric) данные: дым, туман, и т.п.
- Воксельные данные
- Предподсчитанные значения 3D функции

- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура
- 3D: трёхмерная текстура

- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура
- 3D: трёхмерная текстура
- 1D array: массив одномерных текстур одинакового размера (не поддерживается в WebGPU)

Массив одномерных текстур

- Набор градиентов цветов



- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура
- 3D: трёхмерная текстура
- 1D array: массив одномерных текстур одинакового размера (не поддерживается в WebGPU)

- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура
- 3D: трёхмерная текстура
- 1D array: массив одномерных текстур одинакового размера (не поддерживается в WebGPU)
- 2D array: массив двумерных текстур одинакового размера

Массив двумерных текстур

- Массив текстур для однотипных объектов, например, вокселей

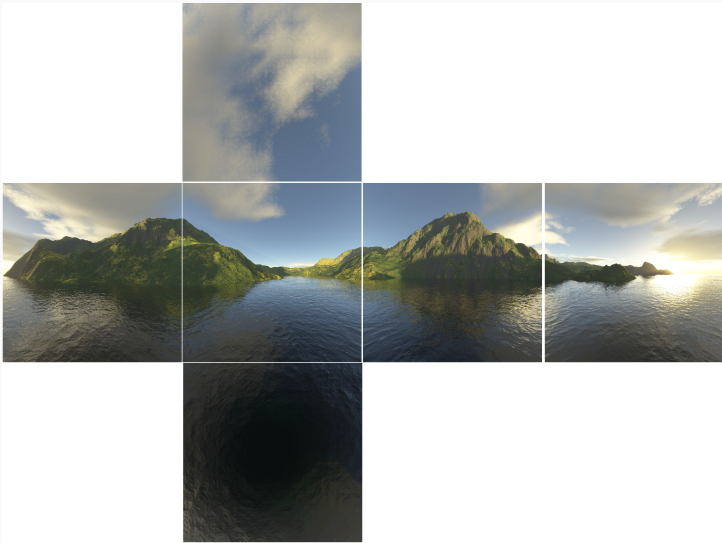


- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура
- 3D: трёхмерная текстура
- 1D array: массив одномерных текстур одинакового размера (не поддерживается в WebGPU)
- 2D array: массив двумерных текстур одинакового размера

- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура
- 3D: трёхмерная текстура
- 1D array: массив одномерных текстур одинакового размера (не поддерживается в WebGPU)
- 2D array: массив двумерных текстур одинакового размера
- Cubemap – текстуры, хранящиеся как 6 граней куба

Субемар-текстура

- Небо (skybox), отражения (environment map, reflection map), предпосчитанные значения функции на сфере



- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура
- 3D: трёхмерная текстура
- 1D array: массив одномерных текстур одинакового размера (не поддерживается в WebGPU)
- 2D array: массив двумерных текстур одинакового размера
- Cubemap – текстуры, хранящиеся как 6 граней куба

- 1D: одномерная текстура (полоска пикселей)
- 2D: двумерная текстура
- 3D: трёхмерная текстура
- 1D array: массив одномерных текстур одинакового размера (не поддерживается в WebGPU)
- 2D array: массив двумерных текстур одинакового размера
- Cubemap – текстуры, хранящиеся как 6 граней куба
- Cubemap array

- Текстура – массив (1D/2D/3D) *пикселей* оперделённого *размера* в определённом *формате*

- Текстура – массив (1D/2D/3D) *пикселей* определённого *размера* в определённом *формате*
- Размер – целочисленный трёхмерный вектор, все компоненты больше нуля

- Текстура – массив (1D/2D/3D) *пикселей* определённого *размера* в определённом *формате*
- Размер – целочисленный трёхмерный вектор, все компоненты больше нуля
- Для array текстур последняя компонента размера – количество элементов (*слоёв, layers*) в массиве

- Текстура – массив (1D/2D/3D) *пикселей* определённого *размера* в определённом *формате*
- Размер – целочисленный трёхмерный вектор, все компоненты больше нуля
- Для array текстур последняя компонента размера – количество элементов (*слоёв, layers*) в массиве
- Для cubemap текстур последняя компонента размера – 6

- Текстура – массив (1D/2D/3D) *пикселей* определённого *размера* в определённом *формате*
- Размер – целочисленный трёхмерный вектор, все компоненты больше нуля
- Для array текстур последняя компонента размера – количество элементов (*слоёв, layers*) в массиве
- Для cubemap текстур последняя компонента размера – 6
- Для cubemap array текстур последняя компонента размера – 6 умножить на количество слоёв

- Форматы пикселей похожи на форматы вершин:
 - **R8Unorm** – одна компонента в 1 байт, беззнаковый нормированный ($[0 \dots 255]$ превращается в $[0.0 \dots 1.0]$)
 - **RG16Sint** – две компоненты в 2 байта каждая, знаковый ненормированный (конверсия `int16` -> `float32`)
 - **RGBA32Float** – четыре компоненты в 4 байта каждая, floating-point (значения попадают в шейдер как есть)
 - И т.п.

Текстуры: формат пикселей

- Форматы пикселей похожи на форматы вершин:
 - **R8Unorm** – одна компонента в 1 байт, беззнаковый нормированный ($[0 \dots 255]$ превращается в $[0.0 \dots 1.0]$)
 - **RG16Sint** – две компоненты в 2 байта каждая, знаковый ненормированный (конверсия **int16** -> **float32**)
 - **RGBA32Float** – четыре компоненты в 4 байта каждая, floating-point (значения попадают в шейдер как есть)
 - И т.п.
- Есть особые форматы:
 - **RGB10A2Unorm** – три компоненты по 10 бит, одна в 2 бита, беззнаковые нормированные
 - **RG11B10Ufloat** – две компоненты в 11-битном floating-point, одна в 10-битном
 - **RGB9E5Ufloat** – три компоненты: отдельные мантиссы по 9 бит, общая экспонента в 5 бит

- Есть форматы для буфера глубины / stencil:
 - **Stencil8** – только 8-битный stencil буфер
 - **Depth16Unorm** – 16-битный буфер глубины, беззнаковый нормированный
 - **Depth24Plus** – 24-битный буфер глубины (может быть нормированным или floating-point)
 - **Depth24PlusStencil8** – 24-битный буфер глубины + 8-битный stencil буфер
 - **Depth32Float** – 32-битный floating-point буфер глубины
 - **Depth32FloatStencil8** – 32-битный floating-point буфер глубины + 8-битный stencil буфер
 - И т.п.

Текстуры: формат пикселей

- Есть форматы для буфера глубины / stencil:
 - **Stencil8** – только 8-битный stencil буфер
 - **Depth16Unorm** – 16-битный буфер глубины, беззнаковый нормированный
 - **Depth24Plus** – 24-битный буфер глубины (может быть нормированным или floating-point)
 - **Depth24PlusStencil8** – 24-битный буфер глубины + 8-битный stencil буфер
 - **Depth32Float** – 32-битный floating-point буфер глубины
 - **Depth32FloatStencil8** – 32-битный floating-point буфер глубины + 8-битный stencil буфер
 - И т.п.
- Есть сжатые форматы: **BC***, **ETC***, **EACR***, **ASTC***

Текстуры: формат пикселей

- Есть форматы для буфера глубины / stencil:
 - **Stencil8** – только 8-битный stencil буфер
 - **Depth16Unorm** – 16-битный буфер глубины, беззнаковый нормированный
 - **Depth24Plus** – 24-битный буфер глубины (может быть нормированным или floating-point)
 - **Depth24PlusStencil8** – 24-битный буфер глубины + 8-битный stencil буфер
 - **Depth32Float** – 32-битный floating-point буфер глубины
 - **Depth32FloatStencil8** – 32-битный floating-point буфер глубины + 8-битный stencil буфер
 - И т.п.
- Есть сжатые форматы: **BC***, **ETC***, **EACR***, **ASTC***
- Есть sRGB-форматы: **RGBA8UnormSrgb**, и т.п.

- При создании текстуры нужно указать

- При создании текстуры нужно указать
 - `dimension` – 1D/2D/3D

- При создании текстуры нужно указать
 - `dimension` – 1D/2D/3D
 - `size` – размер массива пикселей (по трём осям)

- При создании текстуры нужно указать
 - **dimension** – 1D/2D/3D
 - **size** – размер массива пикселей (по трём осям)
 - **usage** – как будет использоваться текстура (набор флагов, аналогично buffer usage)

- При создании текстуры нужно указать
 - **dimension** – 1D/2D/3D
 - **size** – размер массива пикселей (по трём осям)
 - **usage** – как будет использоваться текстура (набор флагов, аналогично `buffer usage`)
 - ...и ещё несколько вещей, о которых узнаем позже :)

- CopySrc/CopyDst – будем копировать данные из текстуры / в текстуру

- `CopySrc/CopyDst` – будем копировать данные из текстуры / в текстуру
- `TextureBinding` – будем читать из шейдера как обычную текстуру (и добавлять в bind группы)

- `CopySrc/CopyDst` – будем копировать данные из текстуры / в текстуру
- `TextureBinding` – будем читать из шейдера как обычную текстуру (и добавлять в bind группы)
- `StorageBinding` – будем читать из шейдера как storage текстуру (и добавлять в bind группы)

- `CopySrc/CopyDst` – будем копировать данные из текстуры / в текстуру
- `TextureBinding` – будем читать из шейдера как обычную текстуру (и добавлять в bind группы)
- `StorageBinding` – будем читать из шейдера как storage текстуру (и добавлять в bind группы)
- `RenderAttachment` – будем рисовать в текстуру с помощью растеризации

- `CopySrc/CopyDst` – будем копировать данные из текстуры / в текстуру
- `TextureBinding` – будем читать из шейдера как обычную текстуру (и добавлять в bind группы)
- `StorageBinding` – будем читать из шейдера как storage текстуру (и добавлять в bind группы)
- `RenderAttachment` – будем рисовать в текстуру с помощью растеризации
- `TransientAttachment` – render attachment, который не нужен после окончания render pass'a

- Сама текстура – только контейнер пикселей, чтобы её использовать в шейдере, нужно создать *texture view*:
`texture.create_view()`

- Сама текстура – только контейнер пикселей, чтобы её использовать в шейдере, нужно создать *texture view*:
`texture.create_view()`
- Описывает, как *интерпретировать* текстуру

- Сама текстура – только контейнер пикселей, чтобы её использовать в шейдере, нужно создать *texture view*:
`texture.create_view()`
- Описывает, как *интерпретировать* текстуру
- Может читать 3D текстуру как 3D, 2D array, cubemap, или cubemap array

- Сама текстура – только контейнер пикселей, чтобы её использовать в шейдере, нужно создать *texture view*:
`texture.create_view()`
- Описывает, как *интерпретировать* текстуру
- Может читать 3D текстуру как 3D, 2D array, cubemap, или cubemap array
- Может читать текстуру в другом формате (конвертировать из sRGB-форматов в обычные и наоборот)

- Сама текстура – только контейнер пикселей, чтобы её использовать в шейдере, нужно создать *texture view*:
`texture.create_view()`
- Описывает, как *интерпретировать* текстуру
- Может читать 3D текстуру как 3D, 2D array, cubemap, или cubemap array
- Может читать текстуру в другом формате (конвертировать из sRGB-форматов в обычные и наоборот)
- Может читать только depth или stencil часть depth+stencil формата

- Сама текстура – только контейнер пикселей, чтобы её использовать в шейдере, нужно создать *texture view*:
`texture.create_view()`
- Описывает, как *интерпретировать* текстуру
- Может читать 3D текстуру как 3D, 2D array, cubemap, или cubemap array
- Может читать текстуру в другом формате (конвертировать из sRGB-форматов в обычные и наоборот)
- Может читать только depth или stencil часть depth+stencil формата
- Может читать конкретный диапазон слоёв array-текстуры или mipmap-уровней (о них позже)

- В bind group layout'e:
 - **view_dimension** – 1D / 2D / 2D array / cubemap / etc
 - **sample_type** – **Float** (для floating-point или нормированных текстур), **Sint** для знаковых целых, **Uint** для беззнаковых целых, **Depth** для глубины

- В bind group layout'e:
 - `view_dimension` – 1D / 2D / 2D array / cubemap / etc
 - `sample_type` – `Float` (для floating-point или нормированных текстур), `Sint` для знаковых целых, `Uint` для беззнаковых целых, `Depth` для глубины
- В bind группе:
 - `texture_view`

- В шейдере текстура представлена переменной с особым типом, соответствующим view dimension и sample type

```
// T = f32 / i32 / u32  
texture_1d<T>  
texture_2d<T>  
texture_2d_array<T>  
texture_3d<T>  
texture_cube<T>  
texture_cube_array<T>
```

- `textureLoad(texture, coords, [array_layer], level)` – возвращает `vec4f`/`vec4i`/`vec4u` в зависимости от типа текстуры (даже если в текстуре на самом деле меньше компонент)

- `textureLoad(texture, coords, [array_layer], level)` – возвращает `vec4f`/`vec4i`/`vec4u` в зависимости от типа текстуры (даже если в текстуре на самом деле меньше компонент)
- `coords` – целочисленные координаты пикселя

- `textureLoad(texture, coords, [array_layer], level)` – возвращает `vec4f`/`vec4i`/`vec4u` в зависимости от типа текстуры (даже если в текстуре на самом деле меньше компонент)
- `coords` – целочисленные координаты пикселя
- `array_layer` – номер слоя для array-текстур

- `textureLoad(texture, coords, [array_layer], level)` – возвращает `vec4f`/`vec4i`/`vec4u` в зависимости от типа текстуры (даже если в текстуре на самом деле меньше компонент)
- `coords` – целочисленные координаты пикселя
- `array_layer` – номер слоя для array-текстур
- `level` – номер mipmap-уровня (о них позже)

- Текстура хранит набор дискретных пикселей

- Текстура хранит набор дискретных пикселей
- Вычислять координаты пикселя – неудобно, нужно знать размер текстуры

- Текстура хранит набор дискретных пикселей
- Вычислять координаты пикселя – неудобно, нужно знать размер текстуры
- Хочется обращаться по неким (проинтерполированным из вершинного шейдера) floating-point координатам

- Текстура хранит набор дискретных пикселей
- Вычислять координаты пикселя – неудобно, нужно знать размер текстуры
- Хочется обращаться по неким (проинтерполированным из вершинного шейдера) floating-point координатам
- Хочется сглаживать (интерполировать) значения между пикселями

- Текстура хранит набор дискретных пикселей
- Вычислять координаты пикселя – неудобно, нужно знать размер текстуры
- Хочется обращаться по неким (проинтерполированным из вершинного шейдера) floating-point координатам
- Хочется сглаживать (интерполировать) значения между пикселями
- $\Rightarrow$ режимы фильтрации текстур

- Режимы фильтрации:
 - `Nearest` – использовать ближайший к указанным координатам пиксель
 - `Linear` – использовать линейную/билинейную/трилинейную интерполяцию между соседними 2/4/8 пикселями (для 1D/2D/3D соответственно)
 - **NB:** Для array-текстур номер слоя *всегда ближайший!*

Текстуры: фильтрация

- Режимы фильтрации:
 - `Nearest` – использовать ближайший к указанным координатам пиксель
 - `Linear` – использовать линейную/билинейную/трилинейную интерполяцию между соседними 2/4/8 пикселями (для 1D/2D/3D соответственно)
 - **NB:** Для array-текстур номер слоя *всегда ближайший!*
- Настраиваются отдельно для случая, когда
 - Пиксель текстуры больше пикселя на экране (magnification) – `mag_filter`
 - Пиксель текстуры меньше пикселя на экране (minification) – `min_filter`

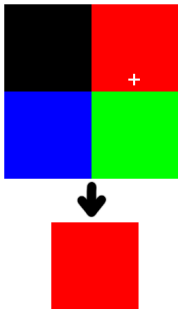
NEAREST



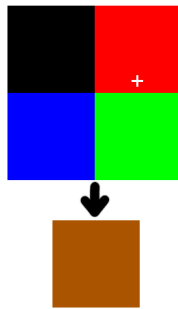
LINEAR



NEAREST



LINEAR



MINIFICATION



MAGNIFICATION



- Когда пиксель текстуры меньше пикселя на экране (minification), текстура выглядит плохо:
 - **Nearest** – часть пикселей текстуры не попадают на экран

- Когда пиксель текстуры меньше пикселя на экране (minification), текстура выглядит плохо:
 - **Nearest** – часть пикселей текстуры не попадают на экран
 - **Linear** – нормально до $\times 2$ уменьшения, потом пиксели тоже не попадают на экран

- Когда пиксель текстуры меньше пикселя на экране (minification), текстура выглядит плохо:
 - **Nearest** – часть пикселей текстуры не попадают на экран
 - **Linear** – нормально до $\times 2$ уменьшения, потом пиксели тоже не попадают на экран
- Решение: *mipmap-уровни*

- Mipmaps – уменьшенные копии текстуры для использования при *минификации*

- Mipmaps – уменьшенные копии текстуры для использования при *минификации*
- Для 2D текстуры $W \times H$ первый уровень имеет размер $\lfloor \frac{W}{2} \rfloor \times \lfloor \frac{H}{2} \rfloor$, второй уровень имеет размер $\lfloor \frac{W}{4} \rfloor \times \lfloor \frac{H}{4} \rfloor$, и т.д.

- Mipmaps – уменьшенные копии текстуры для использования при *минификации*
- Для 2D текстуры $W \times H$ первый уровень имеет размер $\lfloor \frac{W}{2} \rfloor \times \lfloor \frac{H}{2} \rfloor$, второй уровень имеет размер $\lfloor \frac{W}{4} \rfloor \times \lfloor \frac{H}{4} \rfloor$, и т.д.
- Последний mipmap-уровень имеет размер 1×1

- Mipmaps – уменьшенные копии текстуры для использования при *минификации*
- Для 2D текстуры $W \times H$ первый уровень имеет размер $\lfloor \frac{W}{2} \rfloor \times \lfloor \frac{H}{2} \rfloor$, второй уровень имеет размер $\lfloor \frac{W}{4} \rfloor \times \lfloor \frac{H}{4} \rfloor$, и т.д.
- Последний mipmap-уровень имеет размер 1×1
- Для 1D или 3D текстур – аналогично по всем осям

- Mipmaps – уменьшенные копии текстуры для использования при *минификации*
- Для 2D текстуры $W \times H$ первый уровень имеет размер $\lfloor \frac{W}{2} \rfloor \times \lfloor \frac{H}{2} \rfloor$, второй уровень имеет размер $\lfloor \frac{W}{4} \rfloor \times \lfloor \frac{H}{4} \rfloor$, и т.д.
- Последний миптар-уровень имеет размер 1×1
- Для 1D или 3D текстур – аналогично по всем осям
- Для array-текстур – отдельный набор миптар'ов для каждого слоя

- Mipmaps – уменьшенные копии текстуры для использования при *минификации*
- Для 2D текстуры $W \times H$ первый уровень имеет размер $\lfloor \frac{W}{2} \rfloor \times \lfloor \frac{H}{2} \rfloor$, второй уровень имеет размер $\lfloor \frac{W}{4} \rfloor \times \lfloor \frac{H}{4} \rfloor$, и т.д.
- Последний миптар-уровень имеет размер 1×1
- Для 1D или 3D текстур – аналогично по всем осям
- Для array-текстур – отдельный набор миптар'ов для каждого слоя
- Для subemap-текстур – отдельный набор миптар'ов для каждой грани

Mipmaps



- Количество mipmap-уровней указывается в поле `mip_level_count` при создании текстуры

- Количество mipmap-уровней указывается в поле `mip_level_count` при создании текстуры
- Их пиксели нужно загрузить на GPU так же, как саму текстуру, либо сгенерировать уже на GPU (рендерингом или compute шейдерами)

- Количество mipmap-уровней указывается в поле `mip_level_count` при создании текстуры
- Их пиксели нужно загрузить на GPU так же, как саму текстуру, либо сгенерировать уже на GPU (рендерингом или compute шейдерами)
- Режимы фильтрации mipmap-уровней – `mipmap_filter`:
 - **Nearest** – выбирается ближайший mipmap-уровень

- Количество mipmap-уровней указывается в поле `mip_level_count` при создании текстуры
- Их пиксели нужно загрузить на GPU так же, как саму текстуру, либо сгенерировать уже на GPU (рендерингом или compute шейдерами)
- Режимы фильтрации mipmap-уровней – `mipmap_filter`:
 - **Nearest** – выбирается ближайший mipmap-уровень
 - **Linear** – берутся пиксели с двух ближайших уровней и линейно интерполируются

- Размерность текстуры (1D/2D/3D) и тип фильтрации влияют на общее число линейных интерполяций, отсюда названия *билинейная* фильтрация и *трилинейная* фильтрация

Би/трилинейная фильтрация

- Размерность текстуры (1D/2D/3D) и тип фильтрации влияют на общее число линейных интерполяций, отсюда названия *билинейная* фильтрация и *трилинейная* фильтрация
- Часто под трилинейной фильтрацией имеют в виду конкретно 2D текстуру с `min_filter = Linear` и `mipmap_filter = Linear`

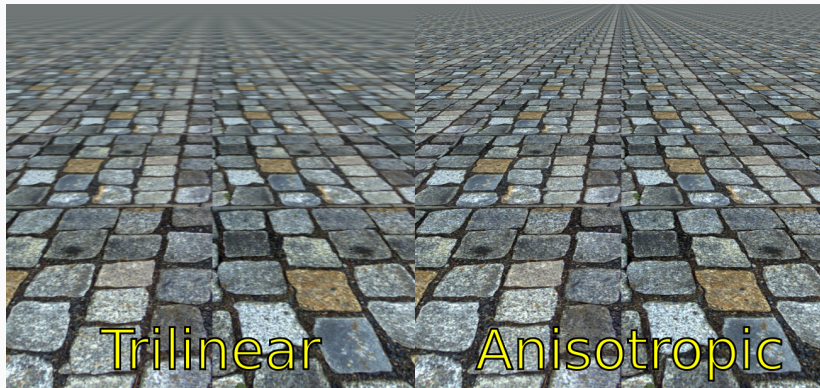
- Текстура после проецирования на экран может быть нормального размера по одной оси, и маленькая по другой

- Текстура после проецирования на экран может быть нормального размера по одной оси, и маленькая по другой
- Большой *mipmap* уровень хорош для одной оси, маленький – для другой; ни один уровень не будет выглядеть хорошо

- Текстура после проецирования на экран может быть нормального размера по одной оси, и маленькая по другой
- Большой mipmap уровень хорош для одной оси, маленький – для другой; ни один уровень не будет выглядеть хорошо
- Решение: *анизотропная фильтрация* – читать текстуру в нескольких местах сразу и усреднять (поддерживается аппаратно)

Анизотропная фильтрация

- Текстура после проецирования на экран может быть нормального размера по одной оси, и маленькая по другой
- Большой mipmap уровень хорош для одной оси, маленький – для другой; ни один уровень не будет выглядеть хорошо
- Решение: *анизотропная фильтрация* – читать текстуру в нескольких местах сразу и усреднять (поддерживается аппаратно)
- Включается настройкой `max_anisotropy` – от 1 до 16 (количество чтений из текстуры, которые делает GPU)



- При чтении из текстуры координаты могут выходить за пределы диапазона

Wrapping mode

- При чтении из текстуры координаты могут выходить за пределы диапазона
- При **Linear**-фильтрации для точек у границы текстуры может не найтись достаточное количество соседних пикселей

Wrapping mode

- При чтении из текстуры координаты могут выходить за пределы диапазона
- При **Linear**-фильтрации для точек у границы текстуры может не найтись достаточное количество соседних пикселей
- $\implies$ **address mode**:
 - **ClampToEdge** – брать пиксель с границы
 - **ClampToBorder** – брать пиксель фиксированного цвета
 - **Repeat** – повторять текстуру
 - **MirrorRepeat** – повторять текстуру, отражая её на каждый повтор

Wrapping mode

- При чтении из текстуры координаты могут выходить за пределы диапазона
- При **Linear**-фильтрации для точек у границы текстуры может не найтись достаточное количество соседних пикселей
- $\implies$ **address mode**:
 - **ClampToEdge** – брать пиксель с границы
 - **ClampToBorder** – брать пиксель фиксированного цвета
 - **Repeat** – повторять текстуру
 - **MirrorRepeat** – повторять текстуру, отражая её на каждый повтор
- Настраивается отдельно для каждой текстурной координаты (U, V, W)

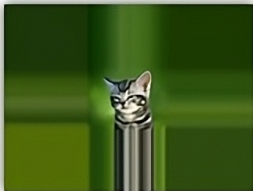
Wrapping



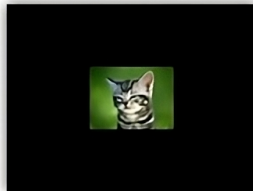
GL_REPEAT



GL_MIRRORED_REPEAT

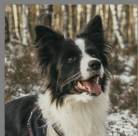


GL_CLAMP_TO_EDGE



GL_CLAMP_TO_BORDER

Wrapping



Border Collie



Clamp Collie



Repeat Collie



Mirror Collie

- Все настройки фильтрации и wrapping'a хранятся не в самой текстуре, а в отдельном объекте – *сэмплере*

- Все настройки фильтрации и wrapping'a хранятся не в самой текстуре, а в отдельном объекте – *сэмплере*
- `device.create_sampler({ addressModeU = Repeat, ... })`

- Все настройки фильтрации и wrapping'a хранятся не в самой текстуре, а в отдельном объекте – *сэмплере*
- `device.create_sampler({ addressModeU = Repeat, ... })`
- Его тоже нужно добавлять в bind группы (и, соответственно, в layout'ы)

- Все настройки фильтрации и wrapping'a хранятся не в самой текстуре, а в отдельном объекте – *сэмплере*
- `device.create_sampler({ addressModeU = Repeat, ... })`
- Его тоже нужно добавлять в bind группы (и, соответственно, в layout'ы)
- **NB:** В некоторых API (OpenGL, Vulkan) можно объединить текстуру и сэмплер в один объект для удобства

- Процесс чтения текстуры с использованием сэмплера (и его настроек) называется *сэмплингом* текстуры

- Процесс чтения текстуры с использованием сэмплера (и его настроек) называется *сэмплингом* текстуры
- Для сэмплинга есть отдельная функция:
`textureSample(texture, sampler, coords, array_layer)`

- Процесс чтения текстуры с использованием сэмплера (и его настроек) называется *сэмплингом* текстуры
- Для сэмплинга есть отдельная функция:
`textureSample(texture, sampler, coords, array_layer)`
- Здесь, `coords` – не номер пикселя, а *текстурные координаты*:

- Процесс чтения текстуры с использованием сэмплера (и его настроек) называется *сэмплингом* текстуры
- Для сэмплинга есть отдельная функция:
`textureSample(texture, sampler, coords, array_layer)`
- Здесь, `coords` – не номер пикселя, а *текстурные координаты*:
 - Для 1D/2D/3D текстур `float/vec2f/vec3f` – нормированные координаты, [0..1] по каждой оси

- Процесс чтения текстуры с использованием сэмплера (и его настроек) называется *сэмплингом* текстуры
- Для сэмплинга есть отдельная функция:
`textureSample(texture, sampler, coords, array_layer)`
- Здесь, **coords** – не номер пикселя, а *текстурные координаты*:
 - Для 1D/2D/3D текстур **float/vec2f/vec3f** – *нормированные координаты*, [0..1] по каждой оси
 - Для subemap текстур **vec3f** – вектор направления из центра куба: возвращается значение из пересечения луча с поверхностью куба

- Процесс чтения текстуры с использованием сэмплера (и его настроек) называется *сэмплингом* текстуры
- Для сэмплинга есть отдельная функция:
`textureSample(texture, sampler, coords, array_layer)`
- Здесь, **coords** – не номер пикселя, а *текстурные координаты*:
 - Для 1D/2D/3D текстур **float/vec2f/vec3f** – *нормированные координаты*, [0..1] по каждой оси
 - Для subemap текстур **vec3f** – вектор направления из центра куба: возвращается значение из пересечения луча с поверхностью куба
 - Частое название в случае 2D текстур: *UV-координаты*

- `texture_2d: vec2(0.6, 0.5)` – пиксель чуть правее центра текстуры

Текстурные координаты

- `texture_2d: vec2(0.6, 0.5)` – пиксель чуть правее центра текстуры
- `texture_cube: vec3(0.1, 1.0, -0.1)` – пиксель на +Y грани куба, около центра грани

Выбор mipmap уровня: shader derivatives

- Как GPU понимает, какой выбрать mipmap-уровень, и происходит ли minification/magnification?

Выбор mipmap уровня: shader derivatives

- Как GPU понимает, какой выбрать mipmap-уровень, и происходит ли minification/magnification? По значениям в *соседних пикселях*

Выбор mipmap уровня: shader derivatives

- Как GPU понимает, какой выбрать mipmap-уровень, и происходит ли minification/magnification? По значениям в *соседних пикселях*
- Фрагментный шейдер всегда запускается на группах 2x2 пикселя (даже если часть из них не были растеризованы)

Выбор mipmap уровня: shader derivatives

- Как GPU понимает, какой выбрать mipmap-уровень, и происходит ли minification/magnification? По значениям в *соседних пикселях*
- Фрагментный шейдер всегда запускается на группах 2x2 пикселя (даже если часть из них не были растеризованы)
- В GLSL есть функции `dpx/dpy` – вычисляют разницу некой величины для пары соседних пикселей в группе 2x2

Выбор mipmap уровня: shader derivatives

- Как GPU понимает, какой выбрать mipmap-уровень, и происходит ли minification/magnification? По значениям в *соседних пикселях*
- Фрагментный шейдер всегда запускается на группах 2x2 пикселя (даже если часть из них не были растеризованы)
- В GLSL есть функции **dpx/dpy** – вычисляют разницу некой величины для пары соседних пикселей в группе 2x2
 - Например, **dpx(value)** – разница между значением **value** в пикселе справа и значением в пикселе слева

Выбор mipmap уровня: shader derivatives

- Как GPU понимает, какой выбрать mipmap-уровень, и происходит ли minification/magnification? По значениям в *соседних пикселях*
- Фрагментный шейдер всегда запускается на группах 2x2 пикселя (даже если часть из них не были растеризованы)
- В GLSL есть функции **dpx/dpy** – вычисляют разницу некой величины для пары соседних пикселей в группе 2x2
 - Например, **dpx(value)** – разница между значением **value** в пикселе справа и значением в пикселе слева
- По **dpx(coords)** и **dpy(coords)** вычисляется mipmap-уровень (где **coords** – текстурные координаты)

Выбор mipmap уровня: shader derivatives

- Как GPU понимает, какой выбрать mipmap-уровень, и происходит ли minification/magnification? По значениям в *соседних пикселях*
- Фрагментный шейдер всегда запускается на группах 2x2 пикселя (даже если часть из них не были растеризованы)
- В GLSL есть функции **dpx/dpy** – вычисляют разницу некой величины для пары соседних пикселей в группе 2x2
 - Например, **dpx(value)** – разница между значением **value** в пикселе справа и значением в пикселе слева
- По **dpx(coords)** и **dpy(coords)** вычисляется mipmap-уровень (где **coords** – текстурные координаты)
- Интерпретируя **dpx/dpy** как аппроксимации производных, можно делать много интересных трюков: восстанавливать нормаль к поверхности по z-буферу, рисовать линии фиксированной толщины даже с перспективной проекцией, и т.п.

- `queue.write_texture(copy_info, data_layout, data, size)`

- `queue.write_texture(copy_info, data_layout, data, size)`
- `data_layout` – описывает, как организованы байты текстуры в памяти:

Текстуры: загрузка данных

- `queue.write_texture(copy_info, data_layout, data, size)`
- `data_layout` – описывает, как организованы байты текстуры в памяти:
 - `bytes_per_row` – количество байт на одну строку текстуры (может включать padding в конце строки)

- `queue.write_texture(copy_info, data_layout, data, size)`
- `data_layout` – описывает, как организованы байты текстуры в памяти:
 - `bytes_per_row` – количество байт на одну строку текстуры (может включать padding в конце строки)
 - `rows_per_image` – количество строк на один 2D слой (имеет смысл для 3D текстур)

- `queue.write_texture(copy_info, data_layout, data, size)`
- `data_layout` – описывает, как организованы байты текстуры в памяти:
 - `bytes_per_row` – количество байт на одну строку текстуры (может включать padding в конце строки)
 - `rows_per_image` – количество строк на один 2D слой (имеет смысл для 3D текстур)
- `copy_info` – описывает, куда нужно записать пиксели:

- `queue.write_texture(copy_info, data_layout, data, size)`
- `data_layout` – описывает, как организованы байты текстуры в памяти:
 - `bytes_per_row` – количество байт на одну строку текстуры (может включать padding в конце строки)
 - `rows_per_image` – количество строк на один 2D слой (имеет смысл для 3D текстур)
- `copy_info` – описывает, куда нужно записать пиксели:
 - `texture` – текстура, куда загружаем пиксели

- `queue.write_texture(copy_info, data_layout, data, size)`
- `data_layout` – описывает, как организованы байты текстуры в памяти:
 - `bytes_per_row` – количество байт на одну строку текстуры (может включать padding в конце строки)
 - `rows_per_image` – количество строк на один 2D слой (имеет смысл для 3D текстур)
- `copy_info` – описывает, куда нужно записать пиксели:
 - `texture` – текстура, куда загружаем пиксели
 - `mip_level` – в какой mipmap-уровень

- `queue.write_texture(copy_info, data_layout, data, size)`
- `data_layout` – описывает, как организованы байты текстуры в памяти:
 - `bytes_per_row` – количество байт на одну строку текстуры (может включать padding в конце строки)
 - `rows_per_image` – количество строк на один 2D слой (имеет смысл для 3D текстур)
- `copy_info` – описывает, куда нужно записать пиксели:
 - `texture` – текстура, куда загружаем пиксели
 - `mip_level` – в какой mipmap-уровень
 - `origin` – координаты левого нижнего угла (можно загрузить часть текстуры в произвольном месте)

- `queue.copy_buffer_to_texture(...)` – скопировать данные из буфера в текстуру

- `queue.copy_buffer_to_texture(...)` – скопировать данные из буфера в текстуру
- Полезно для асинхронной загрузки текстур (через `mapped buffer`), для сжатия текстур на GPU

- `queue.copy_buffer_to_texture(...)` – скопировать данные из буфера в текстуру
- Полезно для асинхронной загрузки текстур (через `mapped buffer`), для сжатия текстур на GPU
- Для этой функции `bytes_per_row` должен быть кратен 256

Как использовать текстуру для 3D-модели?

- Обычно каждая вершина 3D-модели хранит нормированные 2D текстурные координаты (*UV-развёртка*)

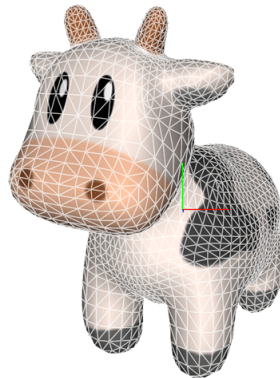
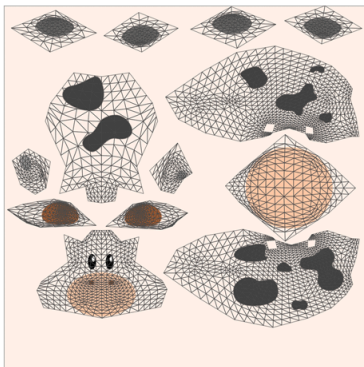
Как использовать текстуру для 3D-модели?

- Обычно каждая вершина 3D-модели хранит нормированные 2D текстурные координаты (*UV-развёртка*)
- Они передаются из вершинного шейдера во фрагментный и интерполируются

Как использовать текстуру для 3D-модели?

- Обычно каждая вершина 3D-модели хранит нормированные 2D текстурные координаты (*UV-развёртка*)
- Они передаются из вершинного шейдера во фрагментный и интерполируются
- Интерполированные значения текстурных координат используются во фрагментном шейдере для сэмплинга текстуры

Как использовать текстуру для 3D-модели?



- `queue.write_texture()` ожидает на вход массив ГОТОВЫХ пикселей

- `queue.write_texture()` ожидает на вход массив готовых пикселей
- Обычно изображения хранятся в более сложных форматах: PNG, JPG, и т.д.

- `queue.write_texture()` ожидает на вход массив готовых пикселей
- Обычно изображения хранятся в более сложных форматах: PNG, JPG, и т.д.
- Есть формат Netpbm, хранящий сырые пиксели и простой для чтения/записи

Форматы изображений

- `queue.write_texture()` ожидает на вход массив готовых пикселей
- Обычно изображения хранятся в более сложных форматах: PNG, JPG, и т.д.
- Есть формат Netpbm, хранящий сырые пиксели и простой для чтения/записи
- Есть библиотеки для чтения разных форматов – libpng, libjpeg

Форматы изображений

- `queue.write_texture()` ожидает на вход массив готовых пикселей
- Обычно изображения хранятся в более сложных форматах: PNG, JPG, и т.д.
- Есть формат Netpbm, хранящий сырые пиксели и простой для чтения/записи
- Есть библиотеки для чтения разных форматов – libpng, libjpeg
- `stb_image.h` – single-header библиотека для чтения разных форматов

- `queue.write_texture()` ожидает на вход массив готовых пикселей
- Обычно изображения хранятся в более сложных форматах: PNG, JPG, и т.д.
- Есть формат Netpbm, хранящий сырые пиксели и простой для чтения/записи
- Есть библиотеки для чтения разных форматов – libpng, libjpeg
- `stb_image.h` – single-header библиотека для чтения разных форматов
- Boost GIL (Generic Image Library)

Текстура: полный код (инициализация)

```
texture = device.create_texture({
    dimension: 2D,
    size: [1024, 1024, 1],
    usage: CopyDst | RenderAttachment,
})

queue.write_texture(...)

sampler = device.create_sampler({
    addressModeU: Repeat,
    minFilter: Linear,
    ...
})
```

Текстура: полный код (инициализация)

```
bind_group_layout = device.create_bind_group_layout([
    {
        binding: 0,
        texture: { Float, 2D },
    },
    {
        binding: 1,
        sampler: { Filtering },
    },
])

bind_group = device.create_bind_group(bind_group_layout, [
    {
        binding: 0,
        texture_view: texture.create_view(),
    },
    {
        binding: 1,
        sampler: sampler,
    },
])
```

Текстура: полный код (рендеринг)

```
render_pass.set_bind_group(0, bind_group)
render_pass.set_pipeline(my_pipeline)
render_pass.set_vertex_buffer(...)
render_pass.set_index_buffer(...)
render_pass.draw_indexed(...)
```

Текстура: полный код (шейдер)

```
@group(0) @binding(0) var color_texture: texture_2d<f32>;
@group(0) @binding(1) var linear_sampler: sampler;

struct VertexIn {
    @location(0) position: vec3f,
    @location(1) texcoord: vec2f,
}

struct VertexOut {
    @builtin(position) clip_position: vec4f,
    @location(0) texcoord: vec2f,
}

@vertex
fn vertex_main(in: VertexIn) -> VertexOut {
    return VertexOut(matrix * vec4f(in.position, 1.0), in.texcoord);
}

@fragment
fn fragment_main(in: VertexOut) -> @location(0) vec4f {
    return textureSample(color_texture, linear_sampler, in.texcoord);
}
```

- [khronos.org/opengl/wiki/Texture](https://www.khronos.org/opengl/wiki/Texture)
- [khronos.org/opengl/wiki/Sampler_Object](https://www.khronos.org/opengl/wiki/Sampler_Object)
- [khronos.org/opengl/wiki/Image_Format](https://www.khronos.org/opengl/wiki/Image_Format)
- learnopengl.com/Getting-started/Textures
- opengl-tutorial.org/beginners-tutorials/tutorial-5-a-textured-cube
- [open.gl/textures](https://www.khronos.org/open.gl/textures)